```
host = config[:host].presence || 'localhost'
```

Defined in `active_support/core_ext/object/blank.rb`.

2.3 duplicable?

A few fundamental objects in Ruby are singletons. For example, in the whole life of a program the integer 1 refers always to the same instance:

```
1.object_id                 # => 3
Math.cos(0).to_i.object_id  # => 3
```

Hence, there's no way these objects can be duplicated through `dup` or `clone`:

```
true.dup  # => TypeError: can't dup TrueClass
```

Some numbers which are not singletons are not duplicable either:

```
0.0.clone        # => allocator undefined for Float
(2**1024).clone  # => allocator undefined for Bignum
```

Active Support provides `duplicable?` to programmatically query an object about this property:

```
"foo".duplicable? # => true
"".duplicable?    # => true
0.0.duplicable?   # => false
false.duplicable? # => false
```

By definition all objects are `duplicable?` except `nil`, `false`, `true`, symbols, numbers, class, module, and method objects.

Any class can disallow duplication by removing `dup` and `clone` or raising exceptions from them. Thus only `rescue` can tell whether a given arbitrary object is duplicable. `duplicable?` depends on the hard-coded list above, but it is much faster than `rescue`. Use it only if you know the hard-coded list is enough in your use case.

Defined in `active_support/core_ext/object/duplicable.rb`.

2.4 deep_dup

The `deep_dup` method returns deep copy of a given object. Normally, when you `dup` an object that contains other objects, Ruby does not `dup` them, so it creates a shallow copy of the object. If you have an array with a string, for example, it will look like this:

```
array     = ['string']
duplicate = array.dup

duplicate.push 'another-string'

# the object was duplicated, so the element was added only to the
duplicate
array     # => ['string']
duplicate # => ['string', 'another-string']

duplicate.first.gsub!('string', 'foo')

# first element was not duplicated, it will be changed in both arrays
array     # => ['foo']
duplicate # => ['foo', 'another-string']
```

As you can see, after duplicating the `Array` instance, we got another object, therefore we can modify it and the original object will stay unchanged. This is not true for array's elements, however. Since `dup` does not make deep copy, the string inside the array is still the same object.

If you need a deep copy of an object, you should use `deep_dup`. Here is an example:

```
array     = ['string']
duplicate = array.deep_dup

duplicate.first.gsub!('string', 'foo')
```